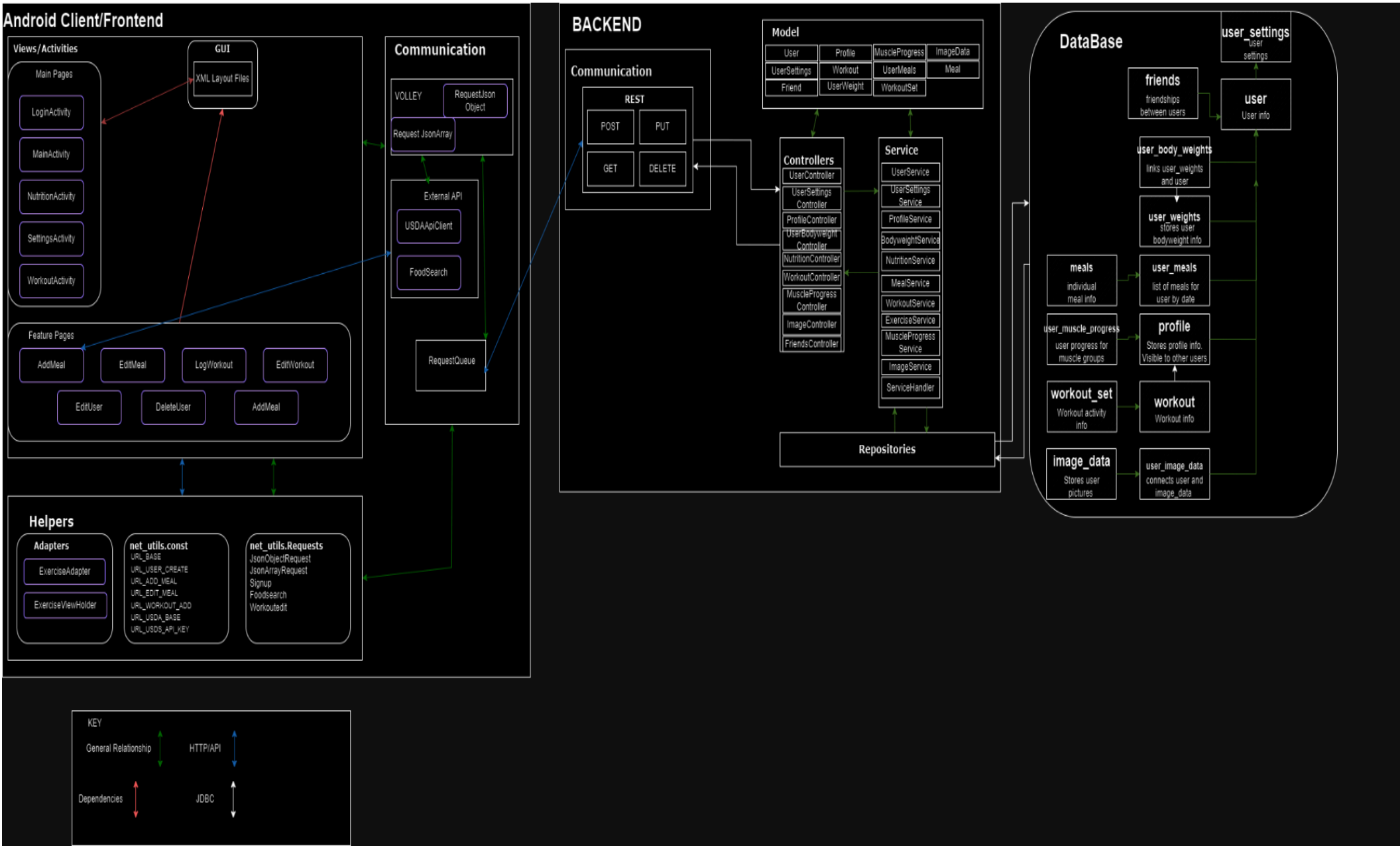
Design Document for Nutri Fit

Group 4_kabir_3

Michael Becker: %50

Nicholas Jacobs: %50


Android Client/Frontend
Views/Activities
GUI
XML Layout Files
Main Pages
LoginActivity
MainActivity
NutritionActivity
SettingsActivity
WorkoutActivity
Feature Pages
AddMeal
EditMeal
LogWorkout
EditWorkout
EditUser
DeleteUser
AddMeal
Communication
VOLLEY
RequestJson Object
Request JsonArray
External API
USDAApiClient
FoodSearch
RequestQueue
Helpers
Adapters
ExerciseAdapter
ExerciseViewHolder
net_utils.const
URL_BASE
URL_USER_CREATE
URL_ADD_MEAL
URL_EDIT_MEAL
URL_WORKOUT_ADD
URL_USDA_BASE
URL_USDS_API_KEY
net_utils.Requests
JsonObjectRequest
JsonArrayRequest
Signup
Foodsearch
Workoutedit
BACKEND
Communication
REST
POST
PUT
GET
DELETE
Model
User
Profile
MuscleProgress
ImageData
UserSettings
Workout
UserMeals
Meal
Friend
UserWeight
WorkoutSet
Controllers
UserController
UserSettings Controller
ProfileController
UserBodyweight Controller
NutritionController
WorkoutController
MuscleProgress Controller
ImageController
FriendsController
Service
UserService
UserSettings Service
ProfileService
BodyweightService
NutritionService
MealService
WorkoutService
ExerciseService
MuscleProgress Service
ImageService
ServiceHandler
Repositories
DataBase
user_settings
User settings
friends
friendships between users
user
User info
user_body_weights
links user_weights and user
user_weights
stores user bodyweight info
meals
individual meal info
user_meals
list of meals for user by date
user_muscle_progress
user progress for muscle groups
profile
Stores profile info. Visible to other users
workout_set
Workout activity info
workout
Workout info
image_data
Stores user pictures
user_image_data
connects user and image_data
KEY
General Relationship
HTTP/API
Dependencies
JDBC

Block Diagram LINK:
https://drive.google.com/file/d/1EE1oFXkQ3WPnuq6ARCujNrK0af_R-jjx/view?usp=sharing
Block Diagram Description:

Frontend (Implemented)
There are multiple pages in our app, the user can navigate around the main pages using a bottom navigation bar, within these pages there are a multitude of other buttons and inputs the user can interact with. In the Nutrition page the user can select to add a meal, this opens up a new page, add meal activity. The user then can select the meal type, input the meal and the macros associated with it. When the user clicks on the "item 1" a dropdown will appear showing the options for meal types. These being Snacks, Breakfast, Lunch and Dinner. When the user searches for a food name, the values are sent to the UsdaApiClient to search the database for the food. It will then autofill the other categories. The user can also manually input data as well. When the Add button is clicked it then sends a POST request to the server. This is an example of the Views/activities communicating with eachother. The external API is then called and automatically fills the POST request with data and it is on to the backend. When the server responds, it is then given a success or fail notification. The page automatically reverts back to the nutrition page and a GET request is automatically sent which fills in the calories completed for that day.

Server:
For our server we utilize Spring Boot with Apache Tomcat through the spring-boot-starter maven dependency which acts as the server for our application. The server consists of several layers:

1. Controller:
   Communicates with the frontend through various request mappings by utilizing the RestController interface. Each controller communicates with its corresponding service layer determined by what entities it is interacting with.
2. Service
   Acts as the middleman between the controller and repositories. This layer handles the business logic of the application, converts Data Transfer objects into entities and

vice versa, and interacts with the corresponding repository by either passing it entities to persist or querying it to retrieve the desired entity.

3. Repository

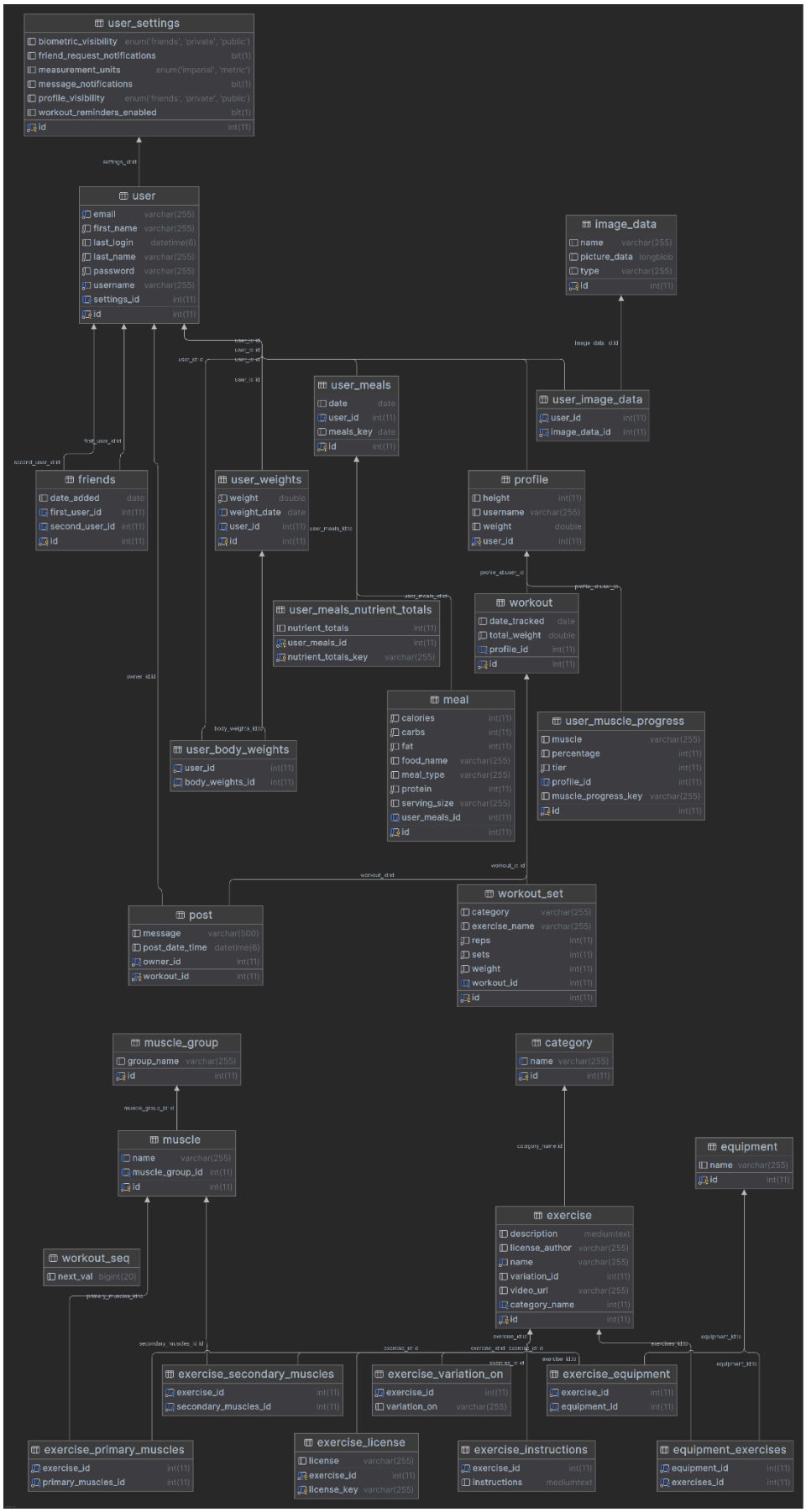
   Communicates with the service layer to either store entities in the database or retrieve them from the database. Communicates with the database to manage entries within related tables.

Database:

For our database, we use MySQL so that we can persist the relationships that different entities have between each other and with their data. The database is mainly comprised of the user table and it's relationship with other tables. The database uses one to one, one to many and many to many relationships to accurately persist the relationships between our entities.

Swagger link: http://coms-3090-058.class.las.iastate.edu:8080/swagger-ui/index.html#/

DB Schema


user_settings
biometric_visibility enum('friends', 'private', 'public')
friend_request_notifications bit(1)
measurement_units enum('imperial', 'metric')
message_notifications bit(1)
profile_visibility enum('friends', 'private', 'public')
workout_reminders_enabled bit(1)
id int(11)
user
email varchar(255)
first_name varchar(255)
last_login datetime(6)
last_name varchar(255)
password varchar(255)
username varchar(255)
settings_id int(11)
id int(11)
image_data
name varchar(255)
picture_data longblob
type varchar(255)
id int(11)
user_meals
date date
user_id int(11)
meals_key date
id int(11)
user_image_data
user_id int(11)
image_data_id int(11)
friends
date_added date
first_user_id int(11)
second_user_id int(11)
id int(11)
user_weights
weight double
weight_date date
user_id int(11)
id int(11)
profile
height int(11)
username varchar(255)
weight double
user_id int(11)
user_meals_nutrient_totals
nutrient_totals int(11)
user_meals_id int(11)
nutrient_totals_key varchar(255)
workout
date_tracked date
total_weight double
profile_id int(11)
id int(11)
meal
calories int(11)
carbs int(11)
fat int(11)
food_name varchar(255)
meal_type varchar(255)
protein int(11)
serving_size varchar(255)
user_meals_id int(11)
id int(11)
user_muscle_progress
muscle varchar(255)
percentage int(11)
tier int(11)
profile_id int(11)
muscle_progress_key varchar(255)
id int(11)
user_body_weights
user_id int(11)
body_weights_id int(11)
post
message varchar(500)
post_date_time datetime(6)
owner_id int(11)
workout_id int(11)
workout_set
category varchar(255)
exercise_name varchar(255)
reps int(11)
sets int(11)
weight int(11)
workout_id int(11)
id int(11)
muscle_group
group_name varchar(255)
id int(11)
category
name varchar(255)
id int(11)
muscle
name varchar(255)
muscle_group_id int(11)
id int(11)
equipment
name varchar(255)
id int(11)
exercise
description mediumtext
license_author varchar(255)
name varchar(255)
variation_id int(11)
video_url varchar(255)
category_name int(11)
id int(11)
workout_seq
next_val bigint(20)
exercise_secondary_muscles
exercise_id int(11)
secondary_muscles_id int(11)
exercise_variation_on
exercise_id int(11)
variation_on varchar(255)
exercise_equipment
exercise_id int(11)
equipment_id int(11)
exercise_primary_muscles
exercise_id int(11)
primary_muscles_id int(11)
exercise_license
license varchar(255)
exercise_id int(11)
license_key varchar(255)
exercise_instructions
exercise_id int(11)
instructions mediumtext
equipment_exercises
equipment_id int(11)
exercises_id int(11)